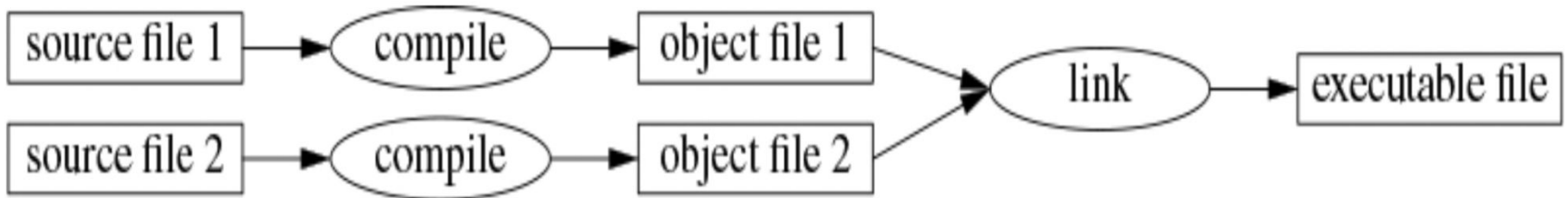


Расширение Основ. C++

Что такое программы на C++?

- C++ – компилируемый язык
- Программа обычно состоит из многих .cpp(.cc) файлов
- Сам выполняемый файл – не портативный, в отличие от исходного кода



Основные сущности C++

- Ядро языка (типы, операторы, циклы)
- Стандартная библиотека (контейнеры, ввод-вывод, алгоритмы)
- Библиотека написана на самом C++, что доказывает мощь C++.

Типизация

- Статическая: типа каждой сущности известен на этапе компиляции
- Тип определяет:
 - Допустимые операции
 - Размещение в памяти

Минимальная программа

```
int main() { }
```

// the minimal C++ program

- main – точка входа
- Возврат int = код завершения
- 0 – успех, не 0 – ошибка

Hello, world

```
1 #include <iostream>
2
3 int main() {
4     std::cout << "Hello, World!\n";
5
6     return 0;
7 }
8
```

- `std::cout`
- `\n` – перевод строки (`std::endl`)

Квадрат

```
1 #include <iostream>           // подключаем библиотеку ввода/вывода
2
3 using namespace std;          // делает имена из std видимыми без std::
4 double square(double x) {     // double – значения с плавающей запятой
5     return x*x;
6 }
7
8 void print_square(double x) {
9     cout << "The square of " << x << " is " << square(x) << endl;
10 }
11
12 int main() {
13     print_square(1.234);       // print: the square of 1.234 is 1.52276
14 }
```

~

Квадрат

```
1 #include <iostream>           // подключаем библиотеку ввода/вывода
2
3 using namespace std;          // делает имена из std видимыми без std::
4 double square(double x) {      // double – значения с плавающей запятой
5     return x*x;
6 }
7
8 void print_square(double x) {
9     cout << "The square of " << x << " is " << square(x) << endl;
10 }
11
12 int main() {
13     print_square(1.234);        // print: the square of 1.234 is 1.52276
14 }
```

~

Возвращаемый тип void – указывает, что функция не возвращает значение

Квадрат

```
1 #include <iostream>           // подключаем библиотеку ввода/вывода
2
3 using namespace std;          // делает имена из std видимыми без std::
4 void print_square(double x);
5 double square(double x);
6
7 int main() {
8     print_square(1.234);       // print: the square of 1.234 is 1.52276
9 }
10
11 void print_square(double x) {
12     cout << "The square of " << x << " is " << square(x) << endl;
13 }
14
15 double square(double x) {     // double – значения с плавающей запятой
16     return x*x;
17 }
```

Возвращаемый тип void – указывает, что функция не возвращает значение

Типы данных

Типы данных	Пример	Диапазон (примерный)	Размер (байты)	Комментарий
int	42	-2 147 483 648 ... 2 147 483 647	4	Целые числа
long long	90000000000	-9e18 ... 9e18	8	Очень большие числа
float	3.14	~1e-38 ... 1e38	4	Дробные(мень ше точность)
double	2.718281828	~1e-308 ... 1e308	8	Дробные(боль ше точность)
char	'A'	0 ... 255 (или -128...127)	1	Один символ
bool	true/false	—	1	Логическое
string	"Hello"	—	n(зависит от длины)	Строка

Арифметические операторы

Оператор	Пример	Результат	Комментарий
+	5 + 3	8	Складывает числа. Если применить к строкам ("Привет" + "Мир") — конкатенация (сложение строк).
-	5 - 3	2	Вычитание
*	5 * 3	15	Умножение
/	7 / 2	3	Целочисленное деление (отбрасывается дробная часть). Для double/float даёт дробный результат.
%	7 % 2	1	Остаток от деления. Только для целых чисел.
++	x++	x = x + 1	Увеличивает на 1. Есть постфиксная(x++) и префиксная (++x) формы.
--	x--	x = x - 1	Уменьшает на 1. Аналогично — постфиксная и префиксная форма.

Операторы сравнения

Оператор	Пример	Результат	Комментарий
>	5 > 4	true	Оператор больше
>=	5 >= 5	true	Оператор больше или равно
<	5 < 4	false	Оператор меньше
<=	5 <= 5	true	Оператор меньше или равно
==	2 == 3	false	Оператор сравнения
!=	2 != 3	true	Оператор неравенства

Логические операторы

- Логическое И (&&)
- Логическое ИЛИ (||)
- Оператор отрицания (!)

Инициализация

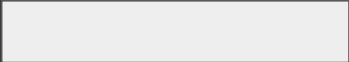
```
double d1 = 2.3;
```

```
double d2 {2.3};
```

```
double d3 = {2.3};
```

```
std::vector<int> v{1, 2, 3, 4, 5, 6, 7, 8, 9};
```

Инициализация

```
int i1 = 7.8;    //   
int i2 {7.8};    // error: floating-point to integer conversion
```

Инициализация

```
int i1 = 7.8;    // i1 станет 7  
int i2 {7.8};   // error: floating-point to integer conversion
```

Область видимости

Scope = участок программы, где имя доступно

Примеры:

- локальная (внутри функции)

Имя, объявленное в функции или в лямбде

- класса (имена-члены)

Это область имен внутри определения класса/структуры (поля, методы, вложенные типы)

- пространства имён

Объявлено в пространстве имен

- глобальная

Имя, объявлено вне какой-нибудь структуры

Время жизни

- Локальные → живут до выхода из блока
- Глобальные/namespace → до конца программы
- Члены класса → до разрушения объекта
- В куче(new) → пока не вызвал delete

const

const

const: означает примерно “Я обещаю не изменять это значение”. Это используется в основном для указания интерфейсов, чтобы данные можно было передавать функциям с помощью указателей и ссылок, не опасаясь их изменения. Компилятор обеспечивает выполнение обещания, данного const. Значение может быть рассчитано в run time

The Null Pointer

```
double* pd = nullptr;  
int x = nullptr; // error: nullptr is a pointer not an integer
```

Мы стараемся гарантировать, что указатель всегда указывает на объект, чтобы его разыменованное было корректным. Когда у нас нет объекта, на который можно указать, или если нам

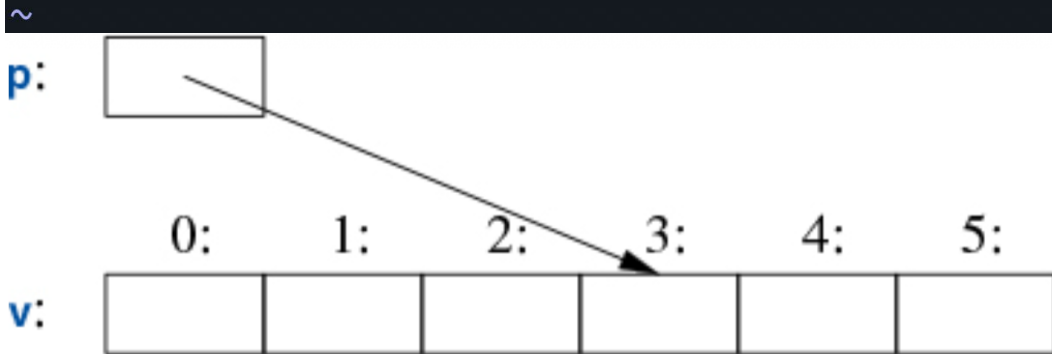
нужно представить понятие «нет доступного объекта» (например, для конца списка), мы присваиваем указателю значение `nullptr` («нулевой указатель»).

Существует только

один общий `nullptr` для всех типов указателей:

Массивы, указатели и ссылки

```
1 char v[6];           // массив из 6 символов
2 char *p = &v[3];      // p указывает на 4-ый элемент массива v
3 char x = *p;          // *p это объект, на который указывает p (разыменование)
```



В выражении префикс унарный * означает “содержимое”,
а префикс унарный & означает “адрес”

Массивы, указатели и ссылки

```
void print2()
{
    int v[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};

    for (auto x : v)                                // for each x in v
        cout << x << '\n';

    for (auto x : {10, 21, 32, 43, 54, 65}) // for each integer in the list
        cout << x << '\n';
    // ...
}
```

Но если все же мы не хотим копировать значения `v` в переменную `x`...

Массивы, указатели и ссылки

```
void increment()
{
    int v[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};

    for (auto& x : v)    // add 1 to each x in v
        ++x;
    // ...
}
```

В этом выражении & означает “ссылка на”. Ссылка похожа на указатель, но ее не нужно разыменовывать, чтобы получить доступ к значению. Так же ссылка не может отсылаться к другому объекту, после ее инициализации.

Массивы, указатели и ссылки

```
void sort(vector<double>& v);           // Сортируем прямо аргумент  
double sum(const vector<double>& v);  // Считаем без копии и без изменений
```

```
T a[n];  // a — массив из n T
```

```
T* p;    // p — указатель на T
```

```
T& r;    // r — ссылка на T
```

```
T f(A);  // f — функция  $A \rightarrow T$ 
```

Присваивание

```
int x = 2;
```

```
int y = 3;
```

```
x = y;
```

// x becomes 3; so we get x==y

x: 2

y: 3

x = y;

x: 3

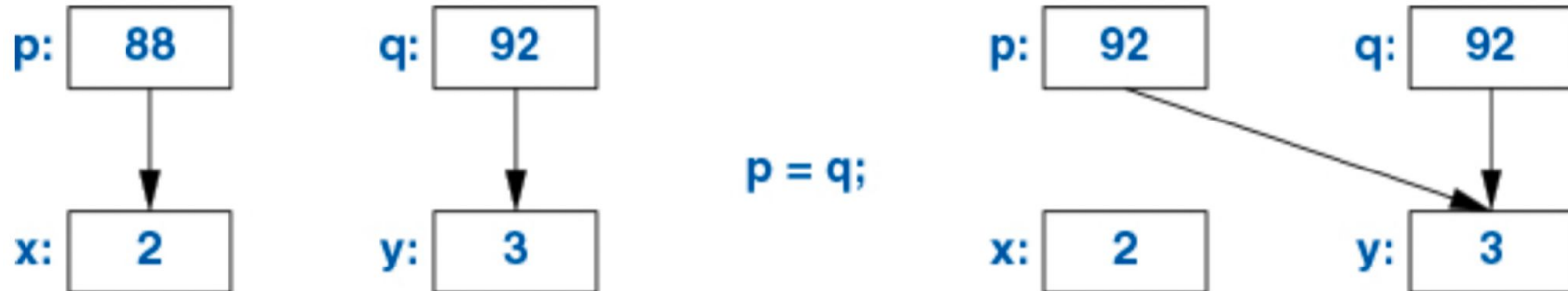
y: 3

Присваивание

```
int x = 2;  
int y = 3;  
int* p = &x;  
int* q = &y;  
p = q;
```

*// p!=q and *p!=*q*

*// p becomes &y; now p==q, so (obviously)*p==*q*



Присваивание

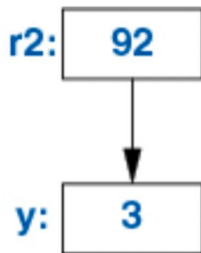
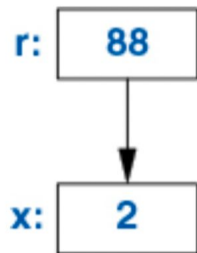
```
int x = 2;
```

```
int y = 3;
```

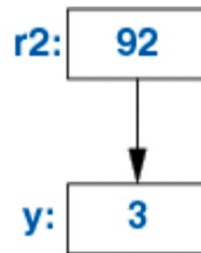
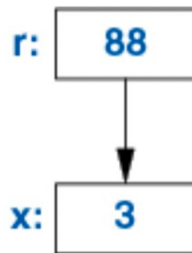
```
int& r = x;    // r refers to x
```

```
int& r2 = y;    // r2 refers to y
```

```
r = r2;        // read through r2, write through r: x becomes 3
```



r = r2;



Статический массив

Объявление и особенности:

```
string bag[10];           // массив на 10 строк
const int SIZE = 100;     // размер должен быть известен на этапе компиляции
string inventory[SIZE];   // фиксированный размер
```



Преимущества статического массива:

- **Высокая скорость** - нет накладных расходов на динамическое выделение
- **Простота** - минимальный синтаксис
- **Предсказуемость** - размер известен заранее



Недостатки:

- **Фиксированный размер** - нельзя изменить во время выполнения
- **Отсутствие проверки границ** - выход за пределы приводит к неопределенному поведению
- **Нет встроенных методов** - размер, добавление, удаление нужно реализовывать вручную

std::vector

Что такое `std::vector`?

`std::vector` — это контейнер стандартной библиотеки C++, который представляет собой **динамический массив** с автоматическим управлением памятью.



Преимущества:

- **Динамический размер** - автоматически расширяется при добавлении элементов
- **Прямой доступ** к элементам по индексу за $O(1)$
- **Автоматическое управление памятью** - не нужно ручное выделение/освобождение
- **Хранение в непрерывной памяти** - эффективное кэширование



Недостатки:

- **Вставка/удаление** в середину за $O(n)$ - требует сдвига элементов
- **Перераспределение памяти** при расширении может быть дорогим

std::vector

Синтаксис:

```
#include <vector>  
using namespace std;
```

```
vector<int> numbers;           // пустой вектор int  
vector<string> words(10);      // вектор из 10 строк  
vector<double> prices = {1.99, 2.49}; // инициализация списком
```

Основные операции

Добавление элементов:

```
vector<string> bag;  
bag.push_back("sword");    // добавить в конец  
bag.emplace_back("potion"); // эффективное создание на месте
```

Доступ к элементам:

```
cout << bag[0];    // быстрый доступ (без проверки границ)  
cout << bag.at(0); // доступ с проверкой границ  
cout << bag.front(); // первый элемент  
cout << bag.back(); // последний элемент
```

Основные операции

Удаление элементов:

```
bag.pop_back();           // удалить последний элемент
bag.erase(bag.begin() + 2); // удалить элемент по индексу 2
bag.clear();              // очистить весь вектор
```

Управление размером:

```
cout << bag.size();       // текущее количество элементов
cout << bag.capacity();   // выделенная память (>= size)
bag.resize(20);           // изменить размер
bag.reserve(100);         // зарезервировать память заранее
```

Основные операции

Итераторы vector:

```
for (auto it = bag.begin(); it != bag.end(); ++it) {  
    cout << *it << endl;  
}
```

```
for (const auto& item : bag) {  
    cout << item << endl;  
}
```